VendorFlow AI

Vendor onboarding, from submission to approval

Complete report: how it is built, how it works, and what happens at every step

Built for	Zamp AI Solutions Analyst case study, Problem Statement 2 (Operations): take a vendor submission and produce an APPROVED, PENDING or REJECTED decision with visible reasoning, and tell the vendor what is still needed.
In one sentence	An eight-stage pipeline, with human review alongside, in which AI only reads documents and writes one sentence, while plain, tested code makes every decision and every step is recorded.
What exists today	A working Streamlit app (live run view, audit dashboard, review queue), a verification engine with 11 checks plus grounding, Gemini document reading with model failover, a SQLite audit trail, ten demo scenarios and 314 automated tests (313 pass and 1 optional cross-check is skipped when its reference library is not installed).
Not yet done	Public deployment, a recorded demo video, and rehearsal of the live interview run.

Contents

1. The problem and what was built
2. Design principles
3. System overview: architecture and stack
4. The pipeline at a glance
5. Step by step: what happens at each stage
6. The checks in detail
7. Findings and the decision rules
8. How AI is used, and how it is kept safe
9. Resubmission, human review and the audit trail
10. The data model
11. The user interface
12. Worked examples from real runs
13. Edge cases and where each is caught
14. Testing and quality
15. Assumptions and known limits
16. Running the demo
17. Glossary

1. The problem and what was built

Before a company can pay a new vendor, someone has to verify the vendor is real and that the details are consistent: company name, tax ID, bank account, and the documents that prove them. In practice vendors send incomplete forms, attach the wrong document, use a bank account in a different name, or reuse another vendor's paperwork. The review is manual, the follow-up is manual, and the only record is somebody's inbox. A bad onboarding can mean payment fraud or a compliance failure.

The case study asks for a process that takes a vendor submission and returns a clear status, with the reasoning visible, and tells the vendor what is needed when something is not approved. VendorFlow AI does this:

- **Input:** a form (company name, country, tax ID, bank account holder and number) plus a tax document PDF, and optionally supporting documents.
- **Process:** read the document, check that what was read is genuinely in it, run eleven checks across the form, the document and the history of earlier vendors, and apply fixed rules.
- **Output:** APPROVED, PENDING or REJECTED, every finding with its evidence, a message for the vendor listing exactly what is needed, and a permanent record of every step.
- **Around it:** a live view that shows each stage as it runs, a dashboard of all runs, a queue where a person can decide the unclear cases, and a resubmission loop that shows what changed.

What the vendor-facing statuses mean

Status	Meaning	Who acts next
APPROVED	Every check passed. The vendor is added to the vendor list so later submissions are checked against it.	Nobody. The vendor gets a confirmation.
PENDING	Something is missing, unclear or unverified. The vendor is told exactly what to provide. Some pending runs are also flagged for a person.	The vendor (resubmit with evidence) and/or a reviewer.
REJECTED	A confirmed disqualifying condition: the tax ID or bank account belongs to a vendor on the blocked list.	Nobody. The vendor gets a neutral notice with no details.

Why rejection is rare on purpose
Most problems (a typo, a missing signature, a name that does not match the bank) are fixable, so they lead to PENDING with a specific request. Rejecting a legitimate vendor over a typo would be a worse mistake than asking one more question, so only a match against a confirmed-blocked vendor rejects automatically.

2. Design principles

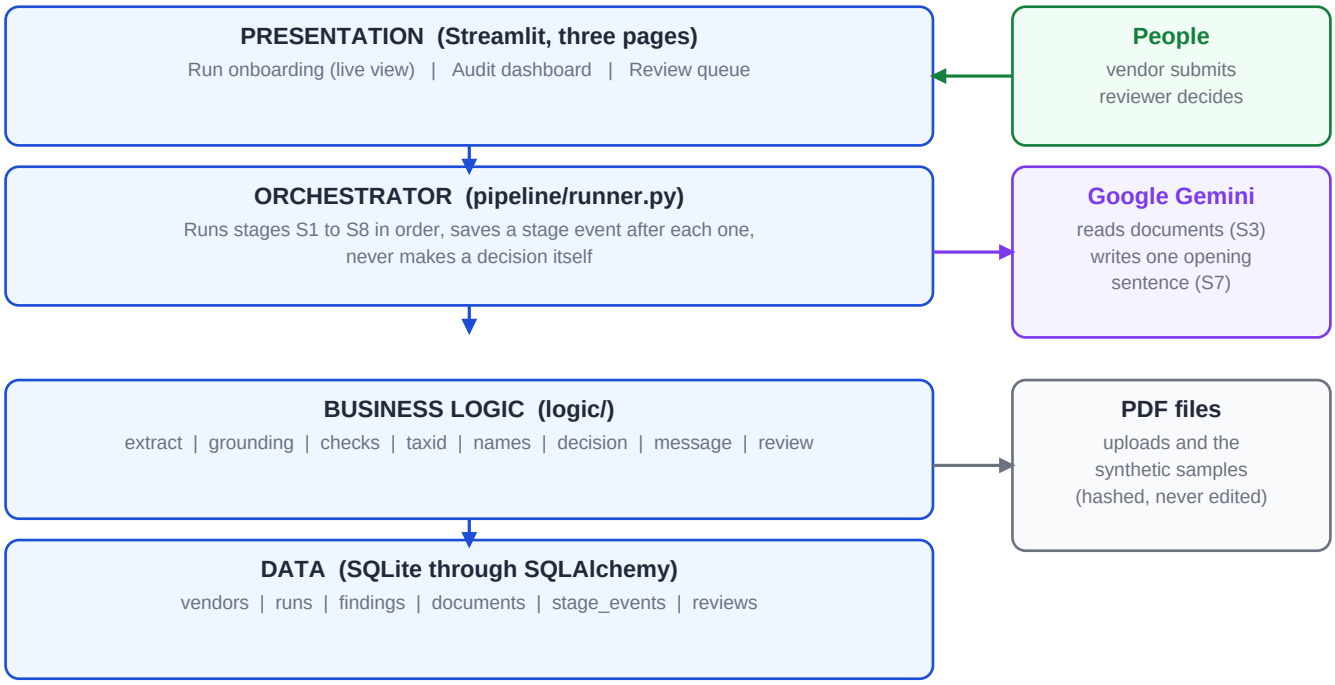
Seven rules shape every part of the system. Each stage in the pipeline exists because of one of them.

#	Principle	What it means in practice
1	**AI reads, code decides**	Gemini turns a document into structured fields and writes one friendly opening sentence. It never approves, rejects, or chooses what the vendor is asked. Plain Python applies fixed rules, so the same findings always give the same status.
2	**Collect everything**	The pipeline never stops at the first problem. A vendor with three issues hears about all three in one message, instead of fixing them one round at a time.
3	**Anything that is not a pass is not a pass**	Only an explicit pass counts toward approval. A failed check, a technical error (such as an API outage) and a skipped check all keep the run PENDING. An outage can never turn into an approval.

#	Principle	What it means in practice
4	**Independent evidence**	What the model read is checked against something the model did not control: the document's own text, or what the vendor typed into the form. The model never sees the form values, so it cannot copy them.
5	**Record as you go**	Each stage writes an event and each finding is saved the moment it exists. If a run crashes halfway, the trail shows exactly where. Events, findings and reviews are only ever added, never edited.
6	**Say only what is safe**	The vendor sees only the pre-written request lines from the findings. Internal evidence (other vendors' ids, fraud signals) never goes into a message or to the model.
7	**People stay in charge**	Unclear cases go to a reviewer. A review is stored next to the automated result and never overwrites it, and confirmed rejections cannot be overridden.

3. System overview: architecture and stack

The system has four layers. Keeping them apart means the whole pipeline can be run and tested without the user interface, and the interface holds no business logic.



Technology choices

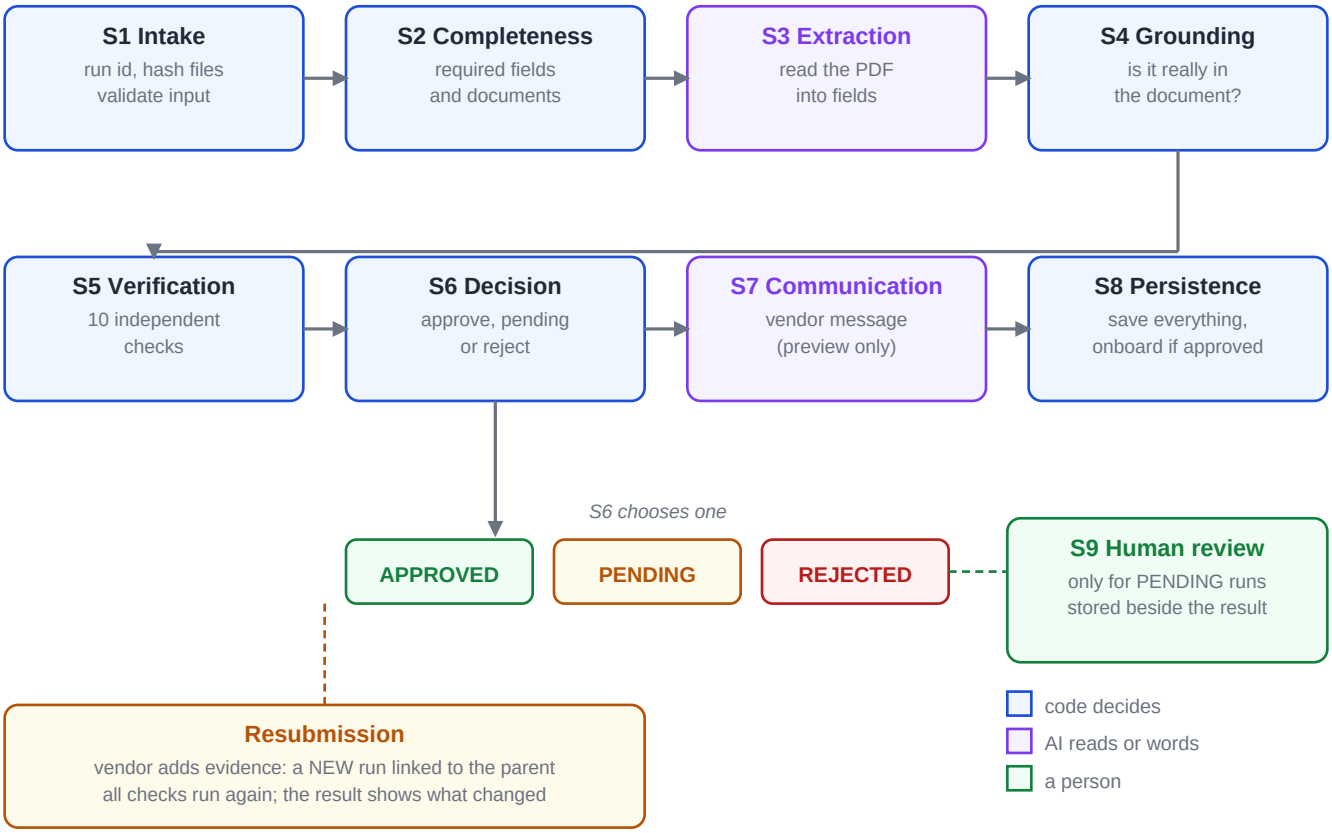
Layer	Choice	Why
Language	Python 3.12	One language for logic, extraction and the interface.
Interface	Streamlit 1.66	A live run view and a dashboard quickly, with real-time updates while the pipeline runs.
Database	SQLite with SQLAlchemy	No setup, enough for demo data, and every run is stored for the audit trail.
PDF reading	pypdf (text layer), PyMuPDF (page images)	Cheap text extraction first; page images only for scans that have no text.
AI	Google Gemini, free tier	Reads documents (text or images) and writes one sentence. Several models are tried in order.
Name matching	rapidfuzz	Fast, well-understood fuzzy string scoring.
Data contracts	Pydantic 2	Strict types where data enters: the submission, the model's output and every finding.
Test data	ReportLab and Pillow	Generate synthetic PDFs, including an image-only scan with no text layer.
Tests	pytest and Streamlit AppTest	314 tests, including headless tests of the interface pages.

Where things live

Folder or file	What it holds
streamlit_app.py, app_pages/	The entry point (navigation, sidebar controls) and the three pages: run.py, dashboard.py, review.py.
ui/	Interface helpers: components (live view, result card, diff, resubmit panel), data (queries and rebuild helpers), presets (the ten scenarios), app_state.
pipeline/runner.py	The orchestrator: runs S1 to S8, writes events, handles resubmission and crashes.
logic/	extract.py (document reading, Gemini client), grounding.py, checks.py (the verification checks), taxid.py (check-digit maths), names.py (matching), decision.py, message.py, review.py, cache.py, models.py (typed contracts), config.py (policy values).
db/	Database models, seeding, reviews, vendor onboarding, document hashing and run-history queries.
samples/, scripts/	Synthetic test PDFs and the scripts that generate them, warm the cache, and smoke-test extraction.
tests/	314 automated tests.

4. The pipeline at a glance

Every submission travels through the same eight stages. Blue stages are plain code, violet stages involve the AI model, and green is a person. After the decision, an unclear run can be reviewed by a person, and a vendor can resubmit.



Stage	Question it answers	Done by	Produces
S1 Intake	What exactly was submitted, and is it a resubmission?	Code	A run record, document hashes
S2 Completeness	Is anything missing from the form or the documents?	Code	One finding listing every gap
S3 Extraction	What does the document say?	pypdf, Gemini	Document type, name, tax ID, signed, date
S4 Grounding	Is what was read really in the document?	Code	One or two grounding findings
S5 Verification	Do the form, the document and the history agree?	Code	Ten findings, each with evidence
S6 Decision	APPROVED, PENDING or REJECTED?	Code	A status and the reasons
S7 Communication	What do we tell the vendor?	Template, Gemini (one sentence)	A message preview
S8 Persistence	Save it, and onboard the vendor if approved.	Code, SQLite	The final record
S9 Human review	Can a person settle the unclear cases?	A reviewer	A review stored beside the result

A run produces 12 findings in the normal case (13 when a supporting document is attached): one from S2, one from S4 (two with a supporting document) and ten from S5.

5. Step by step: what happens at each stage

This section follows one submission through the pipeline. Each card says what happens, why it is designed that way, and what is saved.

S1 Intake	Code
What happens	The input (form fields, tax document path, optional supporting documents) is validated against a strict Submission model. A wrong shape, such as a single path where a list is expected, is rejected before anything is created. A run id (for example **run-0896abbf**) is generated and the raw input is stored. Every uploaded file is hashed (SHA-256 of its bytes) and saved in the documents table. For a resubmission, the parent run's data is the starting point: only the fields the vendor sent replace the old ones, and supporting documents accumulate. A parent must be PENDING and not already closed by a reviewer.
Why it is done this way	Intake must accept bad data, because the system exists to catch it, so it checks shape but applies no business rules. Hashing by content means a file is recognised even if it is renamed.
Stored / emitted	runs row, one documents row per file, S1 events.

S2 Completeness	Code
What happens	Checks that the five form fields (company name, country, tax ID, bank account holder, bank account number) are filled in and that at least one tax document is attached. Every gap is collected into one finding and one request such as *Please provide the missing information: company name, tax registration document*, written with readable labels.
Why it is done this way	A vendor should fix everything in one go. The run does not stop here: later stages that depend on missing input are marked skipped instead of failing, so nothing is reported twice.
Stored / emitted	Finding check_required_fields.

S3 Document extraction	pypdf, Gemini
What happens	For each document: (1) look up the cache by file hash; (2) read the PDF's text layer with pypdf; (3) if there are at least 40 characters of text, send the text to Gemini, otherwise render up to three pages as images and send those; (4) validate the answer strictly (document type, legal name, tax ID, signed, date, entity names); (5) store the result. If Gemini is unavailable and the PDF has text, a regex parser reads it instead and the result says so. A scan with no AI available becomes a technical error. Supporting documents only count if they are a DBA or parent-entity proof that names entities.
Why it is done this way	Extraction turns a document into a claim and nothing more. It never sees the form, so it cannot copy what the vendor typed. A wrong type in its answer (for example the text "true" where a boolean is expected) is rejected, not repaired.
Stored / emitted	extracted_json on the run, S3 event naming the source (gemini-text, gemini-vision or offline_parser).

S4 Grounding	Code
What happens	Checks that the tax ID and legal name the model returned are genuinely in the document. For a PDF with a text layer, each value must appear in that text (ignoring spacing, hyphens, case and accents), whatever the form says. For a scan there is no text to look in, so the value must equal what the vendor typed into the form (after normalisation). Supporting documents get the same treatment for the entity names they list.
Why it is done this way	A wrong value only does harm if it produces a wrong pass. For a scan to pass, the model would have to invent exactly what the vendor typed separately, which is very unlikely. If it does not match, nobody can tell whether the model misread the scan or the vendor made a mistake, so a person looks.
Stored / emitted	Finding check_extraction_grounding, and check_supporting_grounding when a supporting document exists.

S5 Verification	Code
What happens	Ten independent checks run, each recording its result and the evidence behind it: tax ID format, tax ID check digits, document type, signature and date, form tax ID against document tax ID, form name against document name, company name against bank holder name, duplicate tax ID, duplicate bank account, and document reuse. Section 6 describes each. One check crashing becomes an error finding and the rest still run.
Why it is done this way	Each check does one job and never decides. It reports what it found and how serious it is, which keeps every check testable on its own and lets the decision policy change without touching any check.
Stored / emitted	Ten findings, each saved as soon as it exists, plus one S5 progress event per check.

S6 Decision	Code
What happens	A short fixed function reads all findings. Any block gives REJECTED. Otherwise anything that is not an explicit pass gives PENDING. Only when every finding passed is the run APPROVED. It also works out whether a person should look (any finding flagged for review, or a skip with no explaining failure) and lists the exact requests to make of the vendor.
Why it is done this way	About ten lines with no input or output and no AI, so it is fully repeatable and easy to explain: this run is pending because of these two things.
Stored / emitted	The status on the run, and an S6 event that carries it.

S7 Communication	Template, Gemini
What happens	Code builds the message from the request lines of the non-passing findings (duplicates removed). For a PENDING run, Gemini may write one short opening sentence; if its sentence is too long, contains a number or link, or claims an outcome (such as approved or welcome), a plain template sentence is used instead. APPROVED and REJECTED messages are fixed templates that never call the model. Nothing is actually sent.
Why it is done this way	The vendor must receive exactly the list the engine produced. If the model could edit the list, the email and the audit record could disagree. The model is never given the company name or any vendor text.
Stored / emitted	The message on the run, and an S7 event saying whether the model or the template wrote the opening.

S8 Persistence	Code, SQLite
What happens	The message is saved with the run. If the status is APPROVED, the vendor is added to the vendors table, so every later submission is checked against it (a repeat of the same vendor then shows up in the duplicate checks). The final event is written.
Why it is done this way	Approval has a lasting effect, so it is the last thing that happens, after the decision and the message exist.
Stored / emitted	vendors row (if approved), final S8 event.

S9 Human review (outside the pipeline)	A reviewer
What happens	A reviewer opens a PENDING run in the review queue, sees the internal evidence, picks approve, reject or request more information, enters their name and a required note. This adds a review record and an S9 event. An approval adds the vendor to the vendors table.
Why it is done this way	An override is a human judgement made after the fact, so it sits beside the automated result instead of replacing it. The queue shows both.
Stored / emitted	reviews row, S9 event (and a vendors row on approval).

If something fails in the middle

A crash inside any stage is caught: the run is left PENDING, a failed event names the stage, and a finding records the error. The vendor is never approved by accident. The history up to that point is already saved because events and findings are written as they happen.

6. The checks in detail

A normal run produces twelve findings (thirteen with a supporting document): one completeness check, one or two grounding checks and ten verification checks. This table lists all of them, what they compare, and what happens when they do not pass.

Check	What it compares	If it does not pass
Required fields and document (S2)	Five form fields and a document are present	PENDING. Every missing item is listed.
Extraction grounding (S4)	Model's tax ID and name against the document text (text PDFs) or the form (scans)	Text PDF, not found: PENDING. Scan, differs from form: PENDING plus manual review.
Tax ID format	Country pattern: US `NN-NNNNNNN`, UK GB plus 9 digits, Canada 9 digits (optional RT plus 4)	PENDING. Unsupported country goes to manual review.
Tax ID check digits	US EIN prefix on the IRS list; UK VAT mod 97 or 9755; Canadian BN Luhn	PENDING (likely a typo), never rejected.
Document type	Right document for the country: W-9, VAT certificate or CRA letter	PENDING. Asks for the correct document.
Signature and date	Signed (W-9 only) and dated	PENDING. Asks for a complete copy.
Form vs document tax ID	Tax ID typed on the form against the one on the document	PENDING plus manual review (could be a typo, could be fraud).
Form vs document name	Company name on the form against the legal name on the document	PENDING. Manual review if very different. DBA or parent proof can clear it.
Company vs bank holder name	Company name against the bank account holder's name	60 to 89 similar: PENDING, asks for proof. Below 60: PENDING plus manual review.
Duplicate tax ID	Tax ID against all existing vendors	PENDING plus manual review. REJECTED only if the match is a blocked vendor.
Duplicate bank account	Bank account against all existing vendors	PENDING plus manual review if another vendor's. REJECTED only if blocked.
Document reuse	File hash against files from other vendors' runs	PENDING plus manual review.

Name matching

- **Normalising first.** Names are lower-cased, accents and punctuation removed, and trailing legal forms (LLC, Inc, Ltd, Corp, Limited and so on, country-specific) are recognised, so "Acme, Inc.", "ACME Incorporated" and "Acme Inc" are the same. Original names are kept for the evidence.
- **Scoring on the core name.** The legal form is removed before scoring, so "Orion Components" against "Orion Components LLC" scores 100. Word order does not matter. A score of 90 or more passes; 60 to 89 asks for proof; below 60 goes to a person.
- **Entity type is its own warning.** "Orion Components LLC" against "Orion Components Inc" match on the core name but are different kinds of company, so the vendor is asked to confirm the exact registered name.
- **Proof can resolve a mismatch.** A supporting document (DBA registration, certificate of ownership) clears the finding, but only if it names both entities and the names are grounded in the document.
- **Low similarity is never a rejection.** "Acme Technologies" against "Acme Holdings" is not proof of fraud.

Tax ID checks

- **Format** is a pattern per country. Input is forgiven in case and surrounding spaces, and only ASCII digits are accepted.
- **US EIN.** The first two digits must be a prefix the IRS assigns. The list has 83 prefixes taken from the IRS page (last reviewed 09-Apr-2026) and matches an independent reference library exactly.
- **UK VAT.** Digits one to seven are weighted 8, 7, 6, 5, 4, 3, 2, added to the last two digits as a number. The result must divide by 97, or divide by 97 after adding 55 (the "9755" variant). Either passes. This differs from one reference library in two documented edge cases.
- **Canadian BN.** The first nine digits must pass the Luhn check. The program account suffix (such as RT0001) has no check digit.
- **A failure is unresolved, not a rejection**, because a typo is far more likely than a forged number. The vendor is asked to re-check the number.

Duplicates, the blocked list and document reuse

- **Same tax ID or bank account as an existing vendor:** PENDING plus manual review, because it may be a legitimate repeat or a fraud signal. The vendor's message says only that the team needs to confirm some details. The other vendor is never named.
- **Match with a vendor marked blocked:** REJECTED. This is the only automatic rejection, because a blocked entry is a confirmed disqualifier.
- **Document reuse:** every submitted file is hashed. If the same content was used in an earlier, unrelated run for a *different* tax ID, the run goes to manual review (a copied or stolen document). Runs in the same resubmission chain are excluded, so a correction never flags itself.

Grounding rules in one table

PDF type	Value read by the model	Result
Has a text layer	Found in the document text	PASS (whether or not it matches the form; mismatches are the verification checks' job)
Has a text layer	Not found in the document text	PENDING: unverified extraction. The vendor is asked for a clear, text-based copy.
Scan (no text)	Equals the form value, after normalising	PASS with the note "read from scan, confirmed against form"
Scan (no text)	Differs from the form value	PENDING plus manual review. A person decides whether the scan was misread or the form is wrong.

Only names and tax IDs are grounded. Dates and the signature flag are the model's judgements with no literal text to match, so they are covered by the signature and date check instead.

7. Findings and the decision rules

A **finding** is the single unit that drives everything: the decision, the explanation, the vendor message and the dashboard. Each finding has an id, a result, the evidence, an optional request to the vendor, and a flag for human review.

Result	Meaning	Effect on the status
pass	The check succeeded.	Counts toward approval. All findings must be pass for APPROVED.
warn	More evidence is needed (for example a partial name match).	PENDING
fail	Unresolved or incorrect (for example an unsigned document).	PENDING
error	A technical failure, not a verdict (for example the model could not be reached).	PENDING. Never counted as a pass, and a reviewer cannot approve over it.
skipped	The check could not run because its input is missing.	PENDING. A run is never APPROVED while any check is skipped.
block	A confirmed disqualifying condition (a match with a blocked vendor).	REJECTED

The decision function

```
def determine_status(findings):
    results = [f["result"] for f in findings]
    if "block" in results:
        return "REJECTED"
    if not results or any(r != "pass" for r in results):
        return "PENDING"          # skipped, error, warn, fail and unknown values all land here
    return "APPROVED"           # only when every finding is an explicit pass
```

How skipped works

When one problem makes other checks impossible, only the root cause asks the vendor for something. The rest are skipped, so the vendor gets one clear request and the counts stay honest. Examples from the tests:

Situation	The finding that asks	Skipped as a result
A utility bill is uploaded instead of a W-9	Document type (fail): please upload the signed W-9	Signature and date, form vs document name
No document is attached	Required fields and document (fail)	Document type, signature, tax ID and name comparisons, document reuse, grounding
The PDF is too blurry to read	Document type (fail): upload a clear copy	Signature, tax ID and name comparisons
The tax ID format is wrong	Tax ID format (fail)	Tax ID check digits

If a skip appears with no explaining failure next to it, that is unexpected, so the run is flagged for a person instead of waiting silently.

Human review flag

Some findings carry a flag meaning a person should look, because the vendor cannot fix them alone: a tax ID that differs between form and document, a name that is very different, a duplicate of another vendor's tax ID or bank account, a reused document, a scan that does not match the form, an unsupported country, and anything involving the blocked list. The flag puts the run in the review queue.

8. How AI is used, and how it is kept safe

AI is used in exactly two places. Everything else is ordinary code.

Where	What the model does	What it never does
S3 Extraction	Reads a document (text, or page images for a scan) and returns document type, legal name, tax ID, signed, date and entity names as structured data.	Sees the form values, judges whether a vendor looks legitimate, or makes any decision.
S7 Communication	Writes one short, friendly opening sentence for a PENDING message.	Chooses or edits what the vendor is asked, sees the company name, or writes approval or rejection messages.

The Gemini client

- **Several models, tried in order.** The free tier allows about 20 requests per day per model. The client tries gemini-3.1-flash-lite, then gemini-3.5-flash-lite, then gemini-3.5-flash, and skips a model for an hour once its daily quota is used up. Other failures skip a model for a minute.
- **Speed.** These models "think" by default, which took about 57 seconds per call. Setting the thinking level to minimal brings a call to a few seconds, and the client retries without that setting if a model rejects it. Each call has a 45-second limit and one retry.
- **Cache.** A successful model reading is stored by file hash plus a fingerprint of the prompt and schema. A repeat of the same file is instant, costs no quota, and a changed prompt invalidates old answers. Errors and fallback readings are never cached.
- **Fallback.** If the model cannot be reached and the PDF has text, a regex parser reads it and the run says so. A scan with no model becomes a technical error, which keeps the run PENDING.

Safety measures

- **The model never sees the form.** A test captures the real request bodies sent to Gemini and fails if any form value appears. This is what makes the scan rule valid.
- **Strict output validation.** Anything wrong in the answer (a bad type, an unknown document type, extra text where a boolean belongs) is rejected, not repaired.
- **Documents are untrusted data.** The prompt tells the model to report, not obey, instructions found in a document. Even if it were fooled, its output only feeds checks, never decisions.
- **Vendor text never reaches the model.** The opening sentence is generated with no vendor-supplied text. The company name is shortened to one line before it appears in the message.
- **Outcome words are blocked.** An opening containing words such as approved, rejected, accepted or welcome, or any number or link, is replaced by the template.
- **Free tier privacy.** Free-tier content may be used by Google to improve its products, so only synthetic test data should go through the free key.

9. Resubmission, human review and the audit trail

Resubmission

When a vendor sends more evidence, the result is a **new run** that points at the original through its parent id. The original is never changed and never pretended to be approved. Every check runs again, because a new document can create a new conflict. Unchanged documents are read from the cache, so a resubmission is fast.

The result then shows **what changed** compared with the parent, by matching findings by check:

Before	After	Shown as
fail, warn or error	pass	**Resolved**
skipped	pass	**Resolved** (the check could not run before and now passes)
pass	fail, warn, error or skipped	**New** (something got worse)
skipped	fail, warn or error	**New** (the check can run now and found a problem)
a problem	still a problem	**Still open**
pass	pass	Counted as unchanged

Rules: only a PENDING run can be resubmitted, and not one a reviewer has already approved or rejected (a request for more information keeps it open).

Human review

- **Who sees what.** The review queue lists PENDING runs where something needs a person. A reviewer sees the internal evidence, which the vendor never does.
- **Three decisions:** approve, reject, or request more information. A name and a written note are required.
- **Guardrails.** Only PENDING runs can be reviewed. A rejection caused by a blocked vendor cannot be overridden. A reviewer cannot approve a run that has a technical error (re-run it instead), though they can reject it.
- **Never overwrites.** The system's result stays on the run and the review sits beside it. The displayed status is the latest review, and the dashboard shows who decided.
- **An approval onboards the vendor**, so later duplicate checks see them.

The audit trail

- **Events and findings are written as they happen** and only ever added: stage events (with timestamps), findings with evidence, reviews, and document hashes.
- **One row changes in place:** the run record (its status, extracted data, message and vendor id). To keep the history trustworthy, every one of those changes also writes an event.
- **The status can be rebuilt from events alone.** A helper replays a run's events (the decision event, any failure, and any reviewer action) and arrives at the same status as the run row. Tests confirm this for approved, pending, rejected, crashed and reviewer-decided runs.
- **Human actions are events too** (stage S9), so a timeline shows model, code and people in one place.

10. The data model

Six tables. There are deliberately no unique constraints on tax ID or bank account, so duplicate submissions can be stored and investigated.

Table	Holds	Key columns
vendors	Existing and newly onboarded vendors, including blocked ones	legal_name, country, tax_id, bank_account, status (active or blocked)
runs	One row per submission or resubmission	id, parent_run_id, input_json, extracted_json, status, vendor_message, vendor_id, created_at
findings	Every check result with its evidence	run_id, check_id, result, evidence, vendor_request, human_review
stage_events	The timeline: one row per event	run_id, stage (S1 to S9), status (started, progress, completed, failed), detail, created_at
documents	The files a run was submitted with	run_id, role (tax or supporting), filename, sha256
reviews	Human decisions, kept beside the system result	run_id, reviewer, decision, note, created_at

11. The user interface

One Streamlit app with three pages and a sidebar. It is designed so an interviewer can watch the process run and understand every output.

Run onboarding

- **Pick a scenario** from ten presets (below) or start blank. The form fills in; every field can be edited and a document can be chosen from the samples or uploaded (PDF only, up to 10 MB).
- **Run the pipeline.** A live panel lists S1 to S8. Each stage shows a spinner while it works and a green tick with a one-line summary when done; S5 shows each check as it lands.
- **Read the result.** A banner shows the status and the count of checks needing attention (and how many were skipped). Below: a colour-coded table of every check with its evidence and what was asked of the vendor, the message to the vendor with a copy button, and tabs for extracted data, the run timeline and the raw submission.
- **Resubmit with more evidence.** For a pending run, a panel lists what was asked, lets you attach a document and correct form fields, and runs a linked resubmission. The result then shows resolved, still open and new findings.

Audit dashboard

- Tiles for runs, approved, pending, rejected and awaiting review; a chart of runs by final status; a chart of the most frequently flagged checks.
- A history table with status filter and search, showing the system status, the final status, who decided, and the counts of flagged and skipped checks.
- Choose a run (click a row, or use the "open a run by name" selector) for its full detail: findings, message, extracted data, timeline, the diff against its parent, the run chain and any reviews.

Review queue

Runs needing a person, each with the internal evidence and a short form: reviewer name, decision, required note. A history of decisions sits below.

Sidebar

A switch for Gemini (off uses the offline parser), a slider that slows the stages down so each is easy to follow on screen, and a **Reset demo data** button that clears runs and onboarded vendors and restores the seed vendors (needed because an approved vendor is stored and a repeat shows as a duplicate).

The ten demo scenarios

Scenario	What it shows	Expected outcome
Happy path: clean W-9	Every check passes	APPROVED
Edge case 1: parent or DBA name mismatch	Bank account in the parent company's name; then resubmit with the ownership certificate	PENDING, then APPROVED
Edge case 2: right EIN, wrong company	Form and W-9 agree on the tax ID, but the W-9 names another company	PENDING (name check)
Edge case 3: bank account used by another vendor	A shared bank account; the other vendor is not named to the vendor	PENDING plus review
Edge case 4: wrong attachment	A utility bill instead of a W-9; dependent checks skip	PENDING (one clear request)
Unsigned W-9	No signature or date	PENDING
Tax ID differs from the W-9	Form says 12-3456789, document says 12-3456780	PENDING plus review
Scanned W-9 (image only)	Only the vision model can read it; grounded against the form	APPROVED (with AI)
Blocked vendor	Tax ID and bank account of a blocked vendor	REJECTED
Blank form	Everything missing, listed together	PENDING

12. Worked examples from real runs

These examples are taken from real runs of the system on the generated sample documents, with the original evidence text. Document readings came from Gemini (cached), and the message opening uses the template because no live model call was made.

Example 1: parent company name mismatch, then resubmission

Submission. Orion Components LLC, United States, EIN 12-3456789. The bank account is held by *Orion Components Holdings LLC*. The attached document is a signed W-9 for Orion Components LLC.

Run 1 (run-0896abbf): what happened, in order

Stage	Event
S1 Intake	Run created; 1 file hashed.
S2 Completeness	All required fields and a tax document were provided.
S3 Extraction	w9_valid.pdf read by gemini-text: document type w9, name Orion Components LLC, tax ID 12-3456789, signed, dated 2026-10-28.
S4 Grounding	Tax ID and name found in the document text. Pass.
S5 Verification	Nine checks pass. Company vs bank holder name: **warn**. 11 passed, 1 needs attention (counts are cumulative across S2, S4 and S5).
S6 Decision	PENDING; unresolved: check_company_bank_name.
S7 Communication	Opening written by the template.
S8 Persistence	Run saved.

The finding that stopped it

Evidence: 'Orion Components LLC' vs 'Orion Components Holdings LLC' (normalised 'orion components llc' vs 'orion components holdings llc'): core-name similarity 78%. Partial match; relationship evidence required.

Message to the vendor

```
Dear Orion Components LLC,

Thank you for submitting your onboarding details. We need a little more information
before we can finish.

What we need from you:
  - The company name on your submission does not match the bank account holder name.
    Please upload a DBA registration or a document showing the relationship between
    the two entities.

Once we receive this, we will continue the review.
```

Run 2 (run-9776f4e3): the resubmission. The vendor attaches a certificate of ownership. A new run is created, linked to run 1. Both documents are read (the first from the cache). Supporting-document grounding finds both entity names in the certificate's text. The name check now reads: *...core-name similarity 78%. Difference explained by submitted DBA / parent-entity proof.* All thirteen findings pass, the status is APPROVED, and the vendor is onboarded.

What changed since run-0896abbf	
Status	PENDING to APPROVED
Resolved	1: Company vs bank holder name (warn to pass)
Still open	0
New	0 (the new supporting-document grounding check passed, which is not a problem)
Unchanged passes	11

Example 2: a scanned W-9

The PDF is an image with no text layer, so pypdf finds nothing and the pages are sent to Gemini as images (**gemini-vision**). Grounding cannot look in any text, so it compares what the model read with the form: the tax ID and name equal what the vendor typed. The grounding evidence reads *Tax document: read from scan, confirmed against form.* Every other check passes and the run is APPROVED. If the vendor had typed a different tax ID, grounding would instead fail and send the run to a person, because it cannot be known whether the scan was misread or the form was wrong. With the AI switched off the scan cannot be read at all, which is a technical error and keeps the run PENDING.

Example 3: a blocked vendor

Shadow Supplies LLC submits a valid-looking, signed W-9. Eight checks pass. Two block: *Tax ID belongs to blocked vendor id 4* and *Bank account belongs to blocked vendor id 4*. The status is REJECTED and a person need not intervene. The message gives no reason:

```
Dear Shadow Supplies LLC,

Thank you for submitting your onboarding details. After review we are unable to proceed
with onboarding your company at this time.
```

Example 4: several problems at once, then a reviewer

Orion submits an **unsigned** W-9 with a bank account number that already belongs to Northwind Traders. The pipeline reports *both* problems in one run (10 passed, 2 need attention), not one after the other:

Check	Result	Evidence	Asked of the vendor
Signature and date	fail	Document is missing: signature, date.	Upload a complete, signed and dated copy.
Duplicate bank account	fail, review	Bank account already linked to vendor id 1 with a different tax ID; verify ownership.	Our team needs to confirm some registration details (no detail about the other vendor).

The run is PENDING and appears in the review queue. A reviewer, Priya, rejects it with the note *Unsigned form and shared bank account; not onboarding.* This adds a review and an S9 event (`decision=reject; reviewer=Priya`). The run's own status stays PENDING, the displayed status becomes REJECTED (decided by reviewer), and replaying the events alone gives exactly that: system PENDING, final REJECTED, source reviewer.

13. Edge cases and where each is caught

The case study asks for two to four deliberate edge cases. The four below are prepared for the live demo, followed by the other situations the system and its tests cover.

The four demo edge cases

#	Situation	Caught by	Result and why it is handled this way
1	Bank account in a parent company's name, then resubmission	S5 company vs bank holder name; S4 supporting grounding; resubmission	PENDING asking for proof, then APPROVED after proof that names both entities. Shows the follow-up loop and the diff.
2	Right tax ID, wrong company on the document	S5 form vs document name	PENDING. The tax ID check alone would have approved it.
3	Bank account already used by another vendor	S5 duplicate bank account	PENDING plus review. The vendor is not told whose account it is.
4	Wrong attachment (a utility bill)	S5 document type; dependent checks skip	PENDING with one clear request instead of several confusing ones.

Other situations covered

Situation	What happens
Three independent problems in one submission	Three findings, three requests, one message. The pipeline never stops at the first.
Missing fields or no document	One finding lists everything missing; dependent checks skip.
Unsupported country (for example Germany)	PENDING plus manual review: automated checks only cover the US, UK and Canada.
Mistyped tax ID that fails its check digits	PENDING asking the vendor to re-check; never rejected.
Model invents a tax ID that is not in a text PDF	Grounding fails: PENDING. The invented value cannot produce an approval.
Scan whose values differ from the form	PENDING plus review: cannot tell a misread scan from a form mistake.
Same file used for a different tax ID	PENDING plus review: possible copied document.
Same vendor submits again after approval	PENDING: the duplicate checks find the vendor now on file.
Model unavailable, or slow	Next model is tried; then the offline parser for text PDFs; a scan becomes a technical error and stays PENDING.
A check crashes or returns an invalid result	Becomes an error finding; the other checks still run; the run is PENDING.
Document that tries to instruct the model	Treated as data; output is validated; it only feeds checks, never decisions.
Reviewer tries to approve over a technical error	Blocked: re-run the submission instead.

14. Testing and quality

314 automated tests (313 pass; one cross-check against an independent reference library is skipped unless that library is installed, and passed when run with it). No test calls the real AI: a stand-in model is used, and the real request is captured where needed.

Area	Tests	What is verified
Scenarios and decisions	26 + 18 + 21 + 8	Every demo scenario, three problems at once, UK and Canada paths, skipped rules, reviewer rules, name matching, the database and the sample PDFs
Extraction and the model client	23 + 24 + 1	Text vs scan routing, fallback, strict output, retries, model failover, quota, no form values in any request
Grounding	24	All four table rows, supporting documents, invented values, scans against the form
Tax ID maths	38	IRS prefixes, UK VAT variants, Luhn, cross-check with a reference library
Pipeline, diff and reuse	25 + 18 + 19	Stages and events, resubmission, resolved/new/still-open, document reuse and chain exclusion
Contracts and audit	31 + 9	Typed boundaries, invalid input, status rebuilt from events
Interface	29	Every scenario through the real pages, resubmission, diff panel, dashboard, review queue, reset

Beyond the tests, the live path was checked against the real Gemini API on every sample document (including the scan) and in the browser. The UK VAT and Canadian BN algorithms and the IRS prefix list were cross-checked against an independent library on hundreds of thousands of numbers.

15. Assumptions and known limits

These are design assumptions and honest limits. None are hidden; they are the points to be upfront about in the interview.

Assumptions (design choices, not given by the case study)

Area	Assumption
Countries and documents	United States (W-9), United Kingdom (HMRC VAT certificate) and Canada (CRA business number letter). Other countries go to manual review.
Thresholds	Name similarity 90 (pass) and 60 (review) are policy values kept in one config file.
Rejection	Only for a confirmed disqualifier: a match with a blocked vendor. Everything else is PENDING.
Identity	Document reuse treats the form's tax ID as the vendor's identity.
Messages	A preview only; nothing is sent.
Test data	All documents and vendors are synthetic.

Known limits

- **Signature detection.** It relies on what the model or text parser reads. A typed marker counts as a signature, and a real W-9 usually has a drawn one, so this check is not reliable on real forms.
- **The offline parser** is built for the sample layout. On other layouts its checks fail safely (they cannot produce a false pass), but it will often read nothing.
- **No external verification.** Tax IDs are format-checked and checked for valid check digits, not looked up in a registry; bank accounts are not verified with a bank; there is no sanctions screening.
- **UK VAT.** Two documented differences from one reference library remain, because HMRC's own specification was not available to settle them.
- **AI free tier.** About 20 requests per day per model. Caching, model failover and the offline parser keep demos reliable, but a cold run of several new files can be slow.
- **Hosting.** The app is not deployed yet. On a free host the database and cache are temporary and reset on restart.
- **Grounding scope.** Only names and tax IDs are grounded. Dates and the signature flag are not.

Natural next steps

- Registry and sanctions integrations (the check interface already isolates each rule), and bank verification.
- Bank routing, sort code and transit checks with their own check digits (needs extra form fields).
- A document-staleness check, and a blocked-entities table independent of the vendors table.
- A real email sender and vendor authentication.

16. Running the demo

Setup

```
pip install -r requirements.txt
python scripts/make_sample_pdfs.py     # synthetic documents (also made on first launch)
python scripts/warm_cache.py           # read the samples once with Gemini so demos are instant
streamlit run streamlit_app.py

.env (never share it):
GEMINI_API_KEY=...
GEMINI_MODELS=gemini-3.1-flash-lite,gemini-3.5-flash-lite,gemini-3.5-flash
```

A five-minute demo order

Minutes	Do this	Say this
0:00 to 0:45	Reset demo data. Run the happy path.	Each stage runs live; the AI only reads the document, code decides, every step is recorded.
0:45 to 2:15	Run edge case 1, then resubmit with the sample proof.	Pending with a specific request; the resubmission is a new linked run and the diff shows what was resolved.
2:15 to 3:15	Run edge case 2 or 3 and open the findings.	Every check shows its evidence; the other vendor is never named to the vendor.
3:15 to 4:00	Run the scanned W-9.	No text layer, so it is read by vision and grounded against the form.
4:00 to 5:00	Open the dashboard, then the review queue.	History across runs; a person decides the unclear cases; assumptions and next steps.

Tips: warm the cache beforehand, reset the data before each rehearsal (an approved vendor is stored, so a repeat shows as a duplicate), and keep "Use Gemini AI" on for the scanned scenario.

17. Glossary

Term	Meaning
Finding	One check's result with its evidence. The unit that drives the decision, the explanation and the message.
Grounding	Checking that what the model read is really in the document (or, for a scan, equals the form).
Skipped	A check that could not run because its input is missing. Never counts as a pass.
Technical error	A failure of the system itself (for example the model was unreachable). Never counts as a pass.
Block	A confirmed disqualifying condition. The only result that rejects.
Resubmission	A new run, linked to the original by its parent id, after the vendor adds evidence.
Run chain	A run, its parent, and every resubmission descended from the same original.
Stage event	A saved record that a pipeline stage started, progressed, completed or failed.
DBA	Doing business as: a trading name different from the registered legal name.
EIN	US Employer Identification Number, written NN-NNNNNNN.
VAT number	UK VAT registration, written GB plus nine digits.
BN	Canadian Business Number, nine digits, optionally followed by a program account such as RT0001.
Luhn	A common check-digit formula used to validate numbers such as the Canadian BN.
Text layer	Selectable text inside a PDF. A scan has none, only a picture.
Vision	Sending the page images to the model so it can read a document that has no text.